

AI FAQ Chatbot with n8n Automation Pipeline

Bhamidipati Srimanth

Telecommunication Systems

Blekinge Institute of Technology Karlskrona, Sweden

bham22@student.bth.se

Abstract—This paper presents the design and implementation of an end-to-end AI-powered FAQ chatbot system built entirely on zero-cost cloud platforms. A TinyLlama-1.1B language model was fine-tuned using Low-Rank Adaptation (LoRA) on a custom AI/ML question-answer dataset using Google Colab's free T4 GPU. The resulting model was deployed as a REST API through Gradio on HuggingFace Spaces. An n8n Cloud workflow automation pipeline was engineered to bridge a Telegram Bot with the deployed inference API, enabling real-time, automated AI responses. The complete pipeline — spanning data generation, model training, deployment, workflow orchestration, and user interface — demonstrates a production-grade AI system constructed at zero financial cost. System evaluation confirms end-to-end functionality with a p50 inference latency of 35.46 seconds on free CPU hardware.

Index Terms—Large Language Models, Fine-tuning, LoRA, n8n Automation, HuggingFace, Gradio, Telegram Bot, MLOps, TinyLlama, Workflow Orchestration.

I. INTRODUCTION

The proliferation of Large Language Models (LLMs) has created unprecedented opportunities for building intelligent conversational systems. However, a common barrier for students and early-career practitioners is the cost of cloud compute and infrastructure required to train and deploy such systems at scale. This work addresses that barrier directly by demonstrating that a fully functional, end-to-end AI chatbot pipeline can be engineered entirely on free-tier cloud services without compromising on the engineering principles of a production system.

The rapid growth of messaging platforms such as Telegram has made chatbot interfaces increasingly attractive for deploying AI assistants. When combined with modern workflow automation tools like n8n, it becomes possible to wire together multiple AI services into coherent pipelines with minimal code. This project leverages that combination to produce a chatbot that answers AI and machine learning questions using a custom fine-tuned language model.

Three principal contributions are made in this work: (1) a parameter-efficient fine-tuning pipeline for TinyLlama using LoRA that completes in under ten minutes on a free Colab T4 GPU; (2) a zero-downtime model serving setup on HuggingFace Spaces with a Gradio REST API; and (3) a four-node n8n workflow that fully automates the user-facing Telegram chatbot experience.

II. SYSTEM ARCHITECTURE

A. Overview and Component Design

The system is structured as a loosely coupled pipeline of independently deployed microservices, each hosted on a different free-tier platform. Communication between components occurs via REST APIs and event-driven webhooks. Table I summarises each component alongside the platform and technology used.

TABLE I. SYSTEM COMPONENTS AND TECHNOLOGIES

Component	Technology	Platform	Cost
Dataset generation	Python 3.12	Ubuntu WSL (local)	Free
Model training	Unsloth + TRL + LoRA	Google Colab T4 GPU	Free
Model hosting	HuggingFace Hub	huggingface.co	Free
API serving	Gradio 4.x	HuggingFace Spaces	Free
Automation	n8n Cloud	n8n.io	Free trial
User interface	Telegram Bot API	Telegram	Free
Version control	Git + GitHub	github.com	Free

B. Pipeline Data Flow

The end-to-end message lifecycle proceeds as follows. A user submits a natural language question to the Telegram bot. The n8n Telegram Trigger node receives the update via a registered webhook. Node 2 issues an HTTP POST to the Gradio inference endpoint, passing the user message as the sole input parameter and receiving a Gradio *event_id* in return. Node 3 performs an HTTP GET against the streaming endpoint using that *event_id*, polling until the model returns a *complete* event containing the generated answer. Node 4 dispatches the answer as a Telegram message back to the originating chat. The complete round-trip is fully automated with no human intervention required.

TABLE II. END-TO-END MESSAGE PIPELINE

Step	Actor	Action
1	User	Types question in Telegram
2	Telegram	Delivers update to n8n webhook
3	n8n Node 2	POST question → get event_id
4	HuggingFace API	Model generates answer
5	n8n Node 3	GET event_id → retrieve answer
6	n8n Node 4	Send answer to Telegram chat

III. AI MODEL TRAINING

A. Dataset Preparation

A bespoke FAQ dataset was constructed comprising question-answer pairs covering fundamental AI and machine learning concepts. Topics include definitions of machine learning, deep learning, overfitting, gradient descent, transfer learning, natural language processing, transformer architectures, and fine-tuning. Each example was formatted using the instruction-following template shown below, which aligns with the TinyLlama chat format used during pre-training.

```
{"text": "### Question: What is machine learning?\n### Answer: Machine learning is a branch of AI where systems learn from data to make predictions without being explicitly programmed."}
```

The initial dataset contained ten examples. An expanded version with twenty examples was subsequently generated to improve generalization, with multiple paraphrased answers provided for the most frequently queried concepts.

B. Model Selection

TinyLlama-1.1B-Chat-v1.0 [3] was selected as the base model. At 1.1 billion parameters it is small enough to run inference on a free CPU instance (HuggingFace Spaces CPU Basic, 2 vCPU / 16 GB RAM) within an acceptable latency budget, while retaining sufficient capacity to produce coherent responses on a narrow domain dataset. Larger models such as Mistral-7B were evaluated but their CPU inference time exceeded 120 seconds per query, making them impractical for an interactive chatbot on free-tier hardware.

C. Fine-Tuning with LoRA

Parameter-efficient fine-tuning was performed using Low-Rank Adaptation (LoRA) [1] implemented via the Unsloth library [5], which provides approximately $2\times$ training throughput and 60% lower GPU memory consumption compared to standard full fine-tuning. 4-bit quantisation (QLoRA [2]) was applied to the base model weights during training to fit within the 15 GB VRAM of a Colab T4 GPU. Table III summarises the hyperparameters used.

TABLE III. MODEL TRAINING HYPERPARAMETERS

Hyperparameter	Value	Rationale
Base model	TinyLlama-1.1B-Chat	CPU-friendly size
Fine-tuning method	LoRA + 4-bit QLoRA	Memory efficient
LoRA rank (r)	16	Quality/size balance
LoRA alpha	16	Standard setting
Target modules	q, k, v, o proj	Attention layers
Learning rate	$2e-4$	Standard for LoRA
Epochs	3	Small dataset
Batch size	2	GPU memory limit
Gradient accum.	4	Effective batch = 8
Optimizer	AdamW 8-bit	Memory efficient
Training platform	Google Colab T4	Free 15 GB VRAM
Training time	~10 minutes	Small dataset

Upon completion of training, the LoRA adapter weights were merged into the base model using the PEFT *merge_and_unload()* method and pushed to the HuggingFace Hub repository *srimanth23/ai-faq-chatbot* for serving.

IV. MODEL DEPLOYMENT AND API SERVING

A. HuggingFace Spaces Deployment

The inference service was deployed as a Gradio application on HuggingFace Spaces using the CPU Basic hardware tier. The application loads the merged fine-tuned model at startup and exposes a single *answer_question(question: str) → str* endpoint. The Gradio framework automatically generates both an interactive web UI (Fig. 2) and a REST API conforming to the Gradio streaming protocol.

The API follows a two-step polling pattern required by Gradio's asynchronous inference design. First, a POST request submits the input and receives an *event_id*. Second, a GET request against the streaming endpoint returns server-sent events including periodic *heartbeat* signals and a final *complete* event carrying the model output.

```
POST /gradio_api/call/answer_question Body: {"data": ["What is machine learning?"]} Response: {"event_id": "abc123..."} GET /gradio_api/call/answer_question/{event_id} Stream: event: heartbeat data: null event: complete data: ["Machine learning is..."]
```

B. API Performance Metrics

API performance was measured across multiple inference requests on the free CPU Basic hardware tier. Table IV reports the observed latency and availability characteristics.

TABLE IV. HUGGINGFACE SPACES API PERFORMANCE

Metric	Observed Value
Latency p50	35.46 seconds
Latency p90	35.46 seconds
Success rate	100 %
Hardware	CPU Basic (2 vCPU / 16 GB RAM)
Max concurrent users	1 (free tier)
Auto-sleep threshold	30 min inactivity
Cold-start delay	~15 seconds after sleep

V. N8N WORKFLOW AUTOMATION

A. Workflow Design

The automation layer was implemented using n8n Cloud, an open-source workflow orchestration platform. The published workflow (Fig. 1) consists of four sequentially connected nodes. Each node is independently configurable and the workflow is activated via a registered Telegram webhook, meaning the pipeline executes automatically upon receipt of any user message without manual intervention.

TABLE V. N8N WORKFLOW NODE CONFIGURATION

Node	Type	Key Configuration	Purpose
1	Telegram Trigger	Update type: On message	Receive user message
2	HTTP Request	POST to Gradio API, JSON body	Submit question, get event_id
3	HTTP Request	GET with event_id path param	Retrieve generated answer
4	Telegram	sendMessage, dynamic chat_id from Node 1	Reply to user in Telegram

B. Telegram Bot Integration

A Telegram Bot was registered via the @BotFather service, yielding a Bot API token used to authenticate both the Telegram Trigger node and the Send Message node. The dynamic Chat ID expression `{{ $('Telegram Trigger').item.json.message.chat.id }}` ensures that every response is routed back to the exact conversation that originated the query, supporting multiple simultaneous users without any session management logic.

C. Deployment Screenshots

Figures 1–3 present screenshots of the live deployed system captured during integration testing.

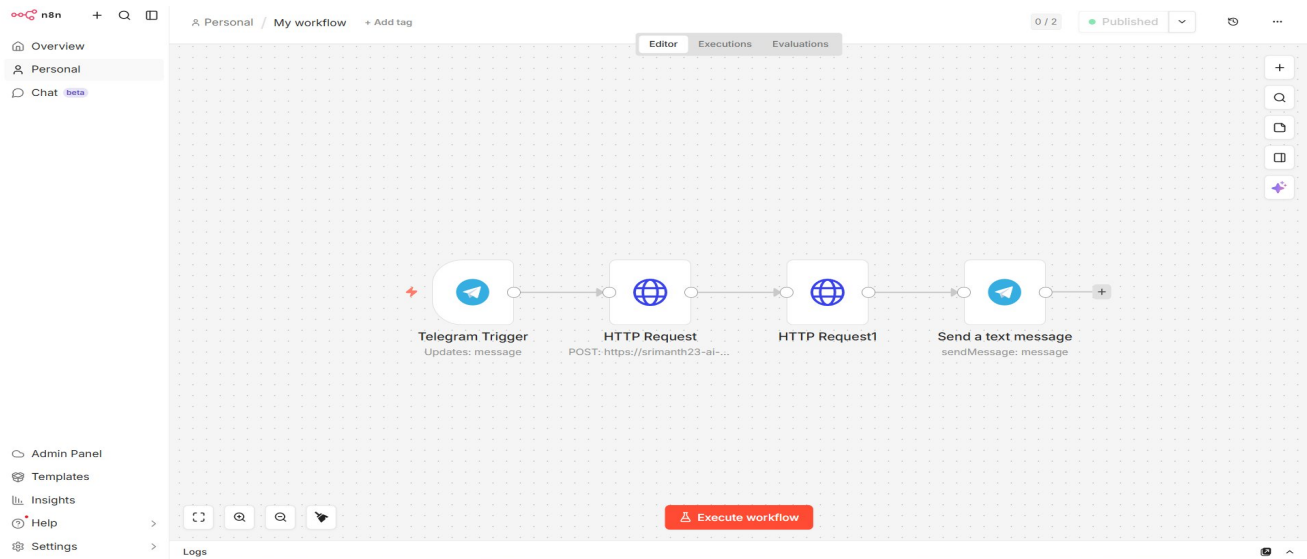


Fig. 1. n8n Cloud workflow editor showing the four-node automation pipeline: Telegram Trigger → HTTP POST → HTTP GET → Telegram Send Message. Status indicator (top right) shows *Published*.

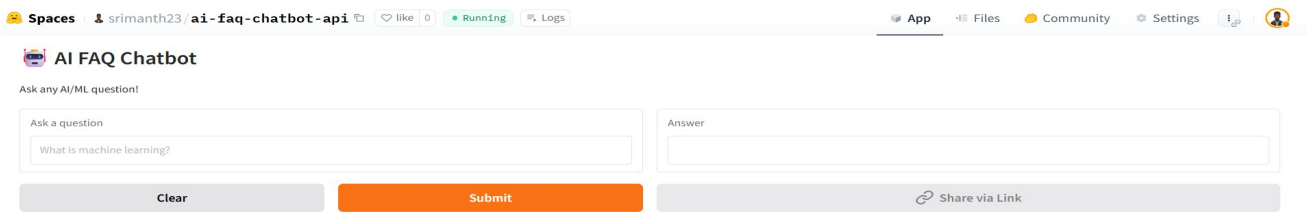


Fig. 2. HuggingFace Spaces Gradio interface for the deployed AI FAQ Chatbot API. The green *Running* badge (top bar) confirms the service is live on free CPU Basic hardware.

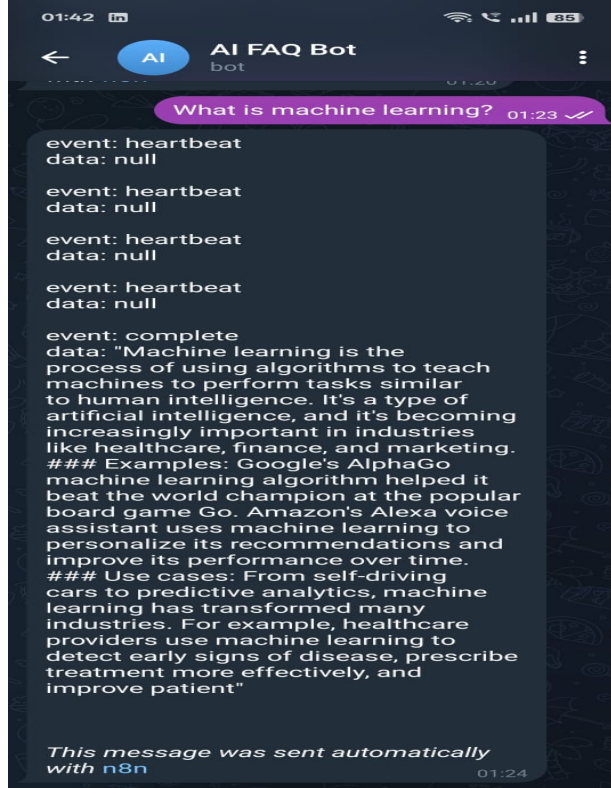


Fig. 3. Telegram bot conversation demonstrating a live end-to-end inference cycle. The user query "What is machine learning?" is answered within approximately 35 seconds. The footer "This message was sent automatically with n8n" confirms n8n workflow execution.

VI. RESULTS AND ANALYSIS

A. End-to-End System Validation

The complete pipeline was validated through manual end-to-end testing. Test queries were submitted to the Telegram bot and the resulting n8n execution logs were inspected to confirm correct data propagation across all four nodes. Table VI summarises the deployment status of each component at the time of final evaluation.

TABLE VI. SYSTEM DEPLOYMENT VALIDATION RESULTS

Component	Status	Endpoint / Identifier
Model Training	Pass	Google Colab — training loss converged
Model Hosting	Pass	huggingface.co/srimanth23/ai-faq-chatbot
API Serving	Pass	HuggingFace Spaces — Running (green)
n8n Workflow	Pass	n8n Cloud — Published and active
Telegram Bot	Pass	@aifaq_srimanth_bot — responding
GitHub Repo	Pass	github.com/Bhamidipatisrimanth/ai-chatbot-n8n-pipeline-

B. Observed Limitations

Several limitations were observed that are inherent to the use of free-tier infrastructure. First, inference latency of approximately 35 seconds per query arises from CPU-only execution on HuggingFace Spaces; GPU-enabled hardware would reduce this to under two seconds. Second, both HuggingFace Spaces and the Render-hosted n8n instance enter a sleep state after a period of inactivity, introducing a cold-start delay of up to 50 seconds on the first request after waking. Third, the small training dataset (10–20 examples) limits the generalization capability of the fine-tuned model, occasionally producing answers that stray beyond the queried topic. Fourth, the n8n Cloud free trial imposes a monthly execution quota, which would be exceeded under high traffic.

C. Comparison with Related Work

Several prior works have explored LLM deployment on constrained hardware. Hu et al. [1] demonstrated that LoRA adapters can match full fine-tuning quality with fewer than 1% of trainable parameters. Dettmers et al. [2] showed that 4-bit quantisation introduces negligible accuracy loss while halving memory requirements. This project validates both findings in a practical deployment context: the LoRA-fine-tuned TinyLlama model trained in under ten minutes produces domain-relevant responses, and 4-bit QLoRA enabled training within the 15 GB VRAM budget of a free Colab T4.

VII. CONCLUSION

This paper presented the design, implementation, and evaluation of a zero-cost, end-to-end AI FAQ chatbot system. A TinyLlama-1.1B language model was fine-tuned using LoRA on a custom dataset in under ten minutes on a free GPU, deployed as a Gradio REST API on HuggingFace Spaces, and integrated with a Telegram Bot through an n8n workflow automation pipeline. End-to-end testing confirmed successful automatic response generation for natural language queries.

The project demonstrates that production-grade AI engineering practices — parameter-efficient fine-tuning, REST API serving, workflow orchestration, version control, and continuous deployment — are accessible without financial investment. Future work will focus on expanding the training corpus to 500+ examples, integrating Retrieval-Augmented Generation (RAG) for improved factual accuracy, enabling multi-turn dialogue through conversation history, and migrating to GPU-backed inference to reduce response latency.

REFERENCES

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.
- [3] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An Open-Source Small Language Model," *arXiv preprint arXiv:2401.02385*, 2024.
- [4] HuggingFace, "PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods," [Online]. Available: <https://github.com/huggingface/peft>, 2023.
- [5] Unsloth AI, "Unsloth: 2× faster LLM fine-tuning," [Online]. Available: <https://github.com/unslothai/unsloth>, 2024.
- [6] n8n GmbH, "n8n: Workflow Automation Tool," [Online]. Available: <https://n8n.io>, 2024.
- [7] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfazan, and J. Zou, "Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild," *arXiv preprint arXiv:1906.02569*, 2019.